

CSE 753 — Imitation Learning and Offline RL: Study Notes

Page 2 — Imitation Learning: Problem Setup

The core idea: we are given a pile of recorded examples of an expert doing a task well (states paired with the actions the expert took), and we want a policy that acts just as well by copying those examples, without ever being told a reward signal or being told the rules of “good” behavior directly.

Mechanism

The data is a set of trajectories

$$\mathcal{D} = \{(s_1, a_1, \dots, s_T)\},$$

where each trajectory is assumed to be a sample from an unknown expert policy π_{expert} .

Symbol	What it means
s_t	the state (or observation) at time step t
a_t	the action taken at time step t
T	the length of one demonstrated trajectory
$\mathcal{D}$	the whole collected dataset of expert trajectories
π_{expert}	the (unknown) rule the expert used to pick actions
π_{θ}	the policy we are training, with learnable parameters θ

We never see π_{expert} itself, only its footprints (the (s, a) pairs it left behind). The goal is to fit π_{θ} so that it performs at roughly the same level as the expert, by mimicking the patterns in $\mathcal{D}$, rather than by being told a reward function. The driving example on the slide makes this concrete: the state is sensor readings from a car, the action is a steering/acceleration command, and the dataset is simply recordings of human drivers.

Page 3 — Behavioral Cloning (“Imitation Learning — Version 0”)

The simplest possible approach: treat this exactly like an ordinary supervised-learning regression problem. For every state in the dataset, make the policy’s predicted action match the expert’s recorded action as closely as possible.

Mechanism

$$\min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(s,a) \in \mathcal{D}} \|a - \hat{a}\|^2, \quad \hat{a} = \pi_{\theta}(s)$$

Symbol	What it means
$ \mathcal{D} $	the number of (s, a) pairs in the dataset
a	the action the expert actually took in state s
$\hat{a}$	the action our deterministic policy predicts for that same s
$\pi_\theta(s)$	a deterministic function: one state in, one action out
$\ \cdot\ ^2$	squared Euclidean distance between the two action vectors

In plain terms: run the state through the network, get a predicted action, measure how far off it was from what the expert did, and nudge the network’s weights (via gradient descent) to shrink that distance, averaged over the whole dataset. Once trained, you simply run π_θ in the world — no further learning. This is behavioral cloning (BC) with a deterministic policy, and it is exactly ordinary L_2 regression from states to actions.

Page 4 — What Can Go Wrong? The Mean-Collapse Problem

If the expert is not perfectly consistent — different demonstrators solve the same situation in different, equally valid ways — L_2 regression does not pick one of those ways. It averages them, and the average can be a terrible, even unsafe, action.

Mechanism

Squared-error loss is minimized, for a fixed state s , by predicting the *mean* of the conditional action distribution $p(a | s)$ in the data. This is a basic fact about squared loss (the same reason the sample mean minimizes sum-of-squared-deviations), not something special to neural networks. So if $p(a | s)$ is multimodal — say, some experts merge left and others stay straight — L_2 regression will output the average of those two very different actions, which is a point that lies *between* the modes and can have almost zero probability under the real data.

Numerical Walkthrough

The slide’s demonstration-distribution plot shows two modes of steering command: “merge left” centered near -2.0 and “stay straight” centered near 0 , with staying straight the more common choice. Suppose, concretely, that 75% of the recorded examples are “stay straight” ($a \approx 0$) and 25% are “merge left” ($a \approx -2$):

$$\mathbb{E}[a | s] = 0.75 \times 0 + 0.25 \times (-2) = -0.5.$$

That matches the dashed “mean of data” line on the slide, sitting at about -0.5 — roughly halfway between the lane markings, straddling both lanes, which is exactly the unsafe, low-probability action the slide warns about. The lesson: *we need to learn beyond the mean*, i.e., we need the policy to represent a distribution that can put probability mass separately on -2 and on 0 , rather than collapsing them into one number.

Page 5 — Neural Networks as Distribution Parameterizations

A neural network that outputs a single action vector can only ever be as expressive as whatever distribution class you decided to wrap around it. Making the network bigger does not fix this — expressive power of the *function* $s \rightarrow$ parameters is a separate question from expressive power of the *distribution class* those parameters define.

Mechanism

Instead of directly outputting an action, the network outputs the parameters of a distribution over actions, $p(a \mid s)$:

- **1D discrete actions** (e.g., Mario: up, down, press A, press B): the network outputs one probability per action, i.e., a full categorical distribution $p(\text{up}), p(\text{down}), \dots$. This is *maximally expressive* for discrete actions — there is no distribution over a finite set of options that a categorical cannot represent exactly.
- **Continuous actions** (e.g., steering angle): the simplest choice is a single Gaussian, so the network outputs $\mu(s)$ and $\sigma(s)$. This is *not very expressive* — a single Gaussian is always unimodal, so it can never represent the two-mode “merge left / stay straight” situation from Page 4 no matter how large the network is.

Contrast to avoid confusing: the expressive power of the mapping $s \rightarrow (\mu, \sigma)$ (which can be made arbitrarily flexible by using a bigger network) is not the same thing as the expressive power of the Gaussian distribution itself (which is fixed and unimodal regardless of network size). A giant network can learn a very accurate $\mu(s)$, but it still cannot describe two separate peaks with only one Gaussian.

Page 6 — Imitation Learning — Version 1: Expressive Policies

The fix for Page 4’s failure mode: instead of regressing to the mean action, train a generative model that assigns high probability to whatever action the expert actually took, using a distribution class expressive enough to have multiple modes.

Mechanism

$$\min_{\theta} -\mathbb{E}_{(s,a) \in \mathcal{D}} [\log \pi_{\theta}(a \mid s)]$$

Symbol	What it means
$\pi_{\theta}(a \mid s)$	the probability (density) our policy assigns to action a in state s
$\log \pi_{\theta}(a \mid s)$	log-likelihood of the expert’s actual action under our policy
$-\mathbb{E}_{(s,a) \in \mathcal{D}} [\cdot]$	average negative log-likelihood over the dataset

This is maximum likelihood estimation: we want π_{θ} to assign as much probability as possible to the actions the expert actually chose, for every (s, a) pair in the dataset. Minimizing the negative log-likelihood is the same optimization as maximizing the likelihood; the sign flip just turns “maximize” into a “minimize” so it fits the usual gradient-descent machinery. Deployment is unchanged: once trained, just sample actions from $\pi_{\theta}(\cdot \mid s)$ (or take its mode) and run the policy.

Page 7 — Gaussian Mixture Models (GMMs)

A GMM is the simplest fix for the “one Gaussian can’t be multimodal” problem: instead of one bump, let the network output several weighted bumps, so it can place separate probability mass on “merge left” and “stay straight” at once.

Mechanism

The network takes state s and outputs the parameters of K Gaussian components:

$$\{w_k(s), \mu_k(s), \Sigma_k(s)\}_{k=1}^K.$$

The probability of an action is the weighted sum of the K component densities:

$$p(a_i | s_i) = \sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i | \mu_k(s_i), \Sigma_k(s_i)).$$

Training maximizes the total log-likelihood of the demonstrated actions, equivalently minimizing the negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \log \left[\sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i | \mu_k(s_i), \Sigma_k(s_i)) \right].$$

Symbol	What it means
K	number of Gaussian components (a hyperparameter you choose ahead of time)
$w_k(s)$	mixture weight of component k in state s ; all $w_k \geq 0$, $\sum_k w_k = 1$
$\mu_k(s)$	mean (center) of component k 's Gaussian bump
$\Sigma_k(s)$	covariance (spread) of component k 's Gaussian bump
$\mathcal{N}(a \mu, \Sigma)$	the Gaussian probability density evaluated at a
N	number of (s, a) training pairs

Tensor shapes / parameter count. If the action is d -dimensional and each component has a diagonal covariance, the network's output layer must produce K scalars for w_k , $K \times d$ numbers for the means, and $K \times d$ numbers for the (diagonal) variances — a total output vector of size $K(1 + 2d)$ (the weights are typically produced as raw scores and passed through a softmax over k so they sum to one and stay non-negative).

Numerical Walkthrough

Let $K = 2$, and suppose for a given state s the network outputs $w_1 = 0.6$, $\mu_1 = -2$, $\sigma_1 = 0.3$ (merge-left component) and $w_2 = 0.4$, $\mu_2 = 0$, $\sigma_2 = 0.3$ (stay-straight component), with 1-D actions. The 1-D Gaussian density is $\mathcal{N}(a | \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(a-\mu)^2/(2\sigma^2)}$. Evaluate the mixture density at $a = -2$ (an exact merge-left action):

$$\mathcal{N}(-2 | -2, 0.3^2) = \frac{1}{0.3\sqrt{2\pi}} e^0 \approx 1.330, \quad \mathcal{N}(-2 | 0, 0.3^2) = \frac{1}{0.3\sqrt{2\pi}} e^{-\frac{4}{0.18}} \approx 1.330 \times e^{-22.2} \approx 0.$$

So $p(a = -2 | s) \approx 0.6(1.330) + 0.4(0) = 0.798$ — almost all the probability mass at $a = -2$ comes from component 1, exactly as intended: a GMM can assign high probability to -2 *and* (separately, via component 2) high probability to 0, something a single Gaussian could never do simultaneously.

Page 8 — Multimodal Actions via Autoregressive Models (Discretize + Chain Rule)

When the action itself has several dimensions (e.g., steering *and* acceleration), a GMM over the whole action vector can get unwieldy. The autoregressive trick: predict one dimension at a time, each time conditioning on the dimensions already chosen, using the chain rule of probability so nothing is lost.

Mechanism

For a multidimensional action $a_i = (a_{i,1}, a_{i,2}, \dots, a_{i,D})$, the chain rule of probability lets us write the exact joint distribution as a product of conditionals:

$$p_\theta(a_i | s_i) = \prod_{d=1}^D p_\theta(a_{i,d} | s_i, a_{i,1:d-1}).$$

Training maximizes the likelihood of the demonstrated actions, i.e., minimizes

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{d=1}^D \log p_\theta(a_{i,d} | s_i, a_{i,1:d-1}).$$

Symbol	What it means
D	number of dimensions in the action (e.g., $D = 2$: steering, acceleration)
$a_{i,1:d-1}$	the first $d - 1$ action dimensions already chosen, for example i
$p_\theta(a_{i,d} s_i, a_{i,1:d-1})$	probability of dimension d given the state and prior dimensions

Each per-dimension conditional is typically represented by *discretizing* that one dimension into bins and outputting a categorical distribution over bins (a classification problem), which is why this is called “discretize + autoregressive.” This is exact — the chain rule is an identity, not an approximation — and the ordering of dimensions is a design choice (a hyperparameter) that does not change how much the model can represent in principle, only how efficiently it does so.

Numerical Walkthrough

Say $D = 2$: dimension 1 is steering, discretized into 3 bins {left, straight, right}, and dimension 2 is acceleration, discretized into 2 bins {low, high}. Suppose the network outputs $p(\text{left}) = 0.3$, $p(\text{straight}) = 0.6$, $p(\text{right}) = 0.1$ for dimension 1. If the sampled/true first action is “left,” then dimension 2’s distribution is recomputed *conditioned on that*, e.g., $p(\text{high} | \text{left}) = 0.8$, $p(\text{low} | \text{left}) = 0.2$ (turning left into oncoming traffic requires more acceleration to clear the gap). The joint probability of the pair (left, high) is then $p(\text{left}) \times p(\text{high} | \text{left}) = 0.3 \times 0.8 = 0.24$ — a single number obtained by multiplying the two conditionals, exactly as the chain-rule formula specifies. Note this joint probability is different from what it would have been if we (incorrectly) assumed independence using the *unconditional* marginal for acceleration, which is why conditioning on prior dimensions matters whenever the dimensions of the action are correlated (as steering and acceleration are here).

Page 9 — Autoregressive Models: Sequential Execution

This slide is the “how do you actually run it” companion to Page 8: the same factorization, but written out as a step-by-step sampling procedure, exactly analogous to how a language model generates a sentence one word at a time.

Mechanism

$$\pi_\theta(a | s) = \prod_{d=1}^D \pi_\theta(a_d | s, a_{<d}), \quad a_{<d} = (a_1, \dots, a_{d-1}).$$

At execution (inference) time, the dimensions are generated one at a time, each sampled from a distribution conditioned on everything sampled so far:

$$a_1 \sim \pi_\theta(a_1 \mid s), \quad a_2 \sim \pi_\theta(a_2 \mid s, a_1), \quad \dots, \quad a_D \sim \pi_\theta(a_D \mid s, a_{1:D-1}).$$

So the complete action is produced by the chain $s \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_D$: each newly sampled dimension is fed back into the model as input for predicting the next dimension. This is structurally identical to a language model predicting one token, feeding that token back in, and predicting the next token — the only difference is that “tokens” here are action-dimension bins rather than words. Because the factorization is exact, this autoregressive policy loses nothing in principle relative to modeling the full joint $\pi_\theta(a \mid s)$ directly; it only trades a hard joint-probability problem for a sequence of easier one-dimension-at-a-time problems.

Page 10 — Decision Transformer

The slide is very terse here (just a diagram and a citation), so this section leans on the original paper, Chen et al., *Decision Transformer: Reinforcement Learning via Sequence Modeling*, NeurIPS 2021 (arXiv:2106.01345), Sections 2–3.

Core Idea

Instead of fitting a value function or computing policy gradients, treat control as a sequence-modeling problem: feed a causally-masked (GPT-style) transformer a sequence of (*return-to-go*, *state*, *action*) triples and train it to predict the next action, the same way a language model predicts the next word.

Mechanism

- **Return-to-go** $\hat{R}_t = \sum_{t'=t}^T r_{t'}$: the sum of *future* rewards from time t onward. This is fed in as an explicit input token, not just used as a training label.
- **Tokenization.** At each timestep t , three raw inputs — $\hat{R}_t$, s_t , a_t — are each passed through their own modality-specific linear embedding layer (one small learned matrix per modality) to become vectors of the same width d_{model} , and a learned positional (timestep) encoding is added to each.
- **Sequence.** These embedded tokens are concatenated in order $(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$, so a trajectory of length T becomes a sequence of $3T$ tokens.
- **Causal transformer.** The sequence is passed through a GPT-style transformer with a causal attention mask, meaning each token can only attend to itself and earlier tokens, never later ones — this is what makes autoregressive generation well-defined.
- **Linear decoder.** At the position of each state token, a linear head reads the transformer’s output and predicts the corresponding action.

Symbol	What it means
$\hat{R}_t$	return-to-go at time t : total reward the trajectory earns from t onward
d_{model}	width of the embedding vectors fed to the transformer
causal mask	attention pattern that blocks a token from seeing future tokens

Tensor shapes (toy example). Suppose $d_{\text{model}} = 4$ and a trajectory has $T = 3$ timesteps. Each of the three modalities (return, state, action) gets its own embedding matrix mapping its raw dimensionality into $\mathbb{R}^4$; after embedding, every token is a length-4 vector, and the full input to the transformer is a sequence of $3T = 9$ such vectors, i.e., a 9×4 matrix (plus positional encodings of the same shape, added elementwise). The linear decoder is a single $4 \times |\mathcal{A}|$ (or $4 \times d_a$ for continuous actions) weight matrix applied at each state-token position.

Why this matters at test time: because return-to-go is an explicit *input*, you can condition generation on a high desired return (e.g., “pretend the rest of this trajectory will earn +100”) and the model, having learned the correlation between stated return-to-go and the actions that achieve it, outputs actions consistent with a high-performing trajectory — effectively turning a hindsight-labeled offline dataset into a controllable, prompt-able policy, entirely through supervised sequence prediction rather than Bellman backups.

Page 11 — “Diffusion Policy” (Flow-Matching Formulation)

Flag/caveat. The slide labels this section “Diffusion Policy,” but the exact training and inference recipe shown — linear interpolation between noise and data, and regressing a velocity field onto $(x_1 - x_0)$ — is the *flow matching* / rectified-flow formulation (Lipman et al., *Flow Matching for Generative Modeling*, 2022), not the original Diffusion Policy paper’s recipe. The original Diffusion Policy (Chi et al., RSS 2023, arXiv:2303.04137) uses a DDPM-style noise-prediction objective with many discrete noise levels $x^k = \sqrt{\bar{\alpha}_k} x^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$ and trains a network to predict the injected noise ϵ . Flow matching instead uses a straight-line path between a noise sample and a data sample and regresses directly onto a constant velocity along that line. Both are “generate an action by transforming noise” methods and both are used under the umbrella term “diffusion policy” in the robotics literature, but they are different training objectives; several recent large-scale robot policies (e.g., Physical Intelligence’s π_0) use exactly the flow-matching recipe shown on this slide rather than the original DDPM recipe.

Mechanism

Training

1. Sample a real action $x_1 \sim \mathcal{D}$ and a noise sample $x_0 \sim \mathcal{N}(0, 1)$.
2. Sample a time $t \sim p(t)$, $t \in [0, 1]$.
3. Linearly interpolate: $x_t = tx_1 + (1 - t)x_0$.
4. Minimize $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$.

Inference

1. Sample noise $x_0 \sim \mathcal{N}(0, 1)$.
2. For $t \in \{0, \delta, 2\delta, \dots, 1 - \delta\}$: $x_{t+\delta} = x_t + v_\theta(x_t, t) \delta$ (an Euler step).
3. Return x_1 .

Symbol	What it means
x_1	a real action sampled from the demonstration data
x_0	a sample of pure Gaussian noise, same shape as an action
t	interpolation time, 0 = pure noise, 1 = real data
x_t	the point on the straight line between x_0 and x_1 at time t
$v_\theta(x_t, t)$	the network’s guess of the velocity (direction and speed) at x_t
δ	the step size used to numerically integrate the ODE at inference

The key insight: along a straight-line path from x_0 to x_1 , the true instantaneous velocity is simply the constant vector $x_1 - x_0$ (the displacement, since position moves at a constant rate along a straight line). So training just means: pick a random point along the line, and teach the network to predict which straight line (i.e., which endpoint pair) that point came from, using only its slope. At inference we don’t know x_1 , so we start from noise x_0 and repeatedly ask the network “which direction should I move,” taking small steps — this numerically solves the differential equation $dx/dt = v_\theta(x, t)$ forward from $t = 0$ to $t = 1$. For imitation learning, the whole process is additionally conditioned on the state s_t (i.e., $v_\theta(x_t, t, s_t)$), so different states can produce different denoising paths and hence different, possibly multimodal, action outputs.

Numerical Walkthrough

Let the action be 1-D. Suppose $x_1 = 2$ (expert action) and $x_0 = -1$ (a sampled noise value), and we train at $t = 0.3$:

$$x_t = 0.3(2) + 0.7(-1) = 0.6 - 0.7 = -0.1, \quad \text{target velocity} = x_1 - x_0 = 2 - (-1) = 3.$$

So the single training example teaches the network: “if you see the point -0.1 at time 0.3 , the correct velocity to output is 3 .” At inference, starting from $x_0 = -1$ with, say, two Euler steps of size $\delta = 0.5$, and supposing the trained network (approximately) outputs the constant velocity 3 at every point on this path:

$$x_{0.5} = x_0 + v_\theta(x_0, 0) \cdot 0.5 = -1 + 3(0.5) = 0.5, \quad x_1 = x_{0.5} + v_\theta(x_{0.5}, 0.5) \cdot 0.5 = 0.5 + 3(0.5) = 2.0,$$

recovering $x_1 = 2$ exactly, because the path really is a straight line with constant velocity 3 in this simplified illustration (with a real trained network the velocity estimate is only approximate, and using more, smaller steps reduces the resulting numerical error).

Page 12 — Problems with Imitation Learning: Compounding Errors

Ordinary supervised learning assumes the inputs you’re tested on come from the same fixed distribution as your training inputs. Imitation learning breaks this assumption: the policy’s own actions determine which states it visits next, so a small mistake changes the input distribution the policy will face going forward.

Mechanism

The diagram shows a demonstrated trajectory $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4$ (teal dots) versus what happens when the learned policy makes small errors $\hat{a}_1, \hat{a}_2, \hat{a}_3$ at each step: the very first small error nudges

the policy to a state slightly off the demonstrated path; because that off-path state was underrepresented (or absent) in the training data, the policy is now *more* likely to err there too, nudging it even further off; the error keeps growing. This is called **compounding error**, and the resulting phenomenon — the states visited by the learned policy $p_\pi(s)$ differing from the states visited by the expert $p^*(s)$ — is called **covariate shift**.

Reference expansion (class transcript, imitation_learning.md, “Compounding Errors” section): the transcript frames this precisely as a break from the i.i.d. assumption of supervised learning — “the inputs to the model are completely independent from the predictions the model makes” in ordinary supervised learning, but “this all changes in imitation learning” because actions drive state transitions. It also notes that even a policy that fits the training distribution *perfectly* can still drift once it makes any small mistake, because perfect in-distribution accuracy says nothing about what happens once you leave the distribution.

Numerical Illustration of Why This Compounds

Suppose the policy has a small per-step probability $\epsilon = 0.05$ of making an error large enough to leave the expert’s state distribution, and once off-distribution it behaves essentially arbitrarily for the rest of a horizon of $T = 50$ steps. The chance of surviving all 50 steps without a single such error is

$$(1 - \epsilon)^T = (0.95)^{50} \approx 0.077,$$

i.e., only about a 7.7% chance of never drifting off. Ross and Bagnell formalized this precisely: for behavioral cloning with per-state error probability ϵ under the expert’s own state distribution, the gap between the expert’s total reward and the learned policy’s total reward can grow *quadratically* in the horizon, $J(\pi^*) - J(\pi) = O(\epsilon T^2)$ (Ross & Bagnell, *Efficient Reductions for Imitation Learning*, 2010). This quadratic-in- T growth is exactly the mathematical signature of compounding error, and it is what the corrective-data methods on the next page are designed to fix.

Page 13 — DAgger: Dataset Aggregation

If compounding error comes from the policy visiting states the expert never demonstrated, the natural fix is to get expert labels *at the states the policy actually visits* — not just the states the expert originally happened to visit.

Mechanism

1. Roll out the current learned policy π_θ to get a trajectory $s'_1, a'_1, \dots, s'_T$.
2. Query the expert for what it would have done at each visited state: $a^* \sim \pi_{\text{expert}}(\cdot \mid s')$.
3. Aggregate these corrections into the dataset: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s', a^*)\}$.
4. Update (retrain or fine-tune) the policy: $\min_\theta \mathcal{L}(\pi_\theta, \mathcal{D})$.

Repeat this loop: each round, the growing dataset $\mathcal{D}$ increasingly covers the states the *current* policy tends to visit (including its own mistakes), not just the states the original expert happened to visit.

Symbol	What it means
s'	a state visited by the current learned policy’s rollout (possibly off-distribution)
a^*	the expert’s action, specifically queried at that visited state s'
$\pi_{\text{expert}}(\cdot \mid s')$	the expert consulted live, not just replayed from old logs

Reference expansion (class transcript). The transcript notes a practical variant of step 2: instead of asking the expert to name the exact action for a given visited state (which can be very hard, e.g., “what steering command exactly?”), you can instead let the expert take full manual control once a mistake is noticed, and log that partial demonstration — this sidesteps having to introspect an exact corrective action. The transcript also flags an important caveat: corrective data must be collected across a *diverse* range of scenarios, or the aggregated dataset overfits to the narrow conditions under which corrections happened (e.g., correcting only on one campus, then failing to generalize elsewhere).

Why this fixes the compounding-error bound. Whereas naive behavior cloning’s error grows as $O(\epsilon T^2)$ (Page 12), Ross, Gordon, and Bagnell showed DAgger’s reduction to online no-regret learning achieves an error bound that is only *linear* in the horizon, $O(\epsilon T)$ (Ross, Gordon & Bagnell, *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*, AISTATS 2011) — directly because the training distribution now matches the distribution the deployed policy will actually encounter. The tradeoff is cost: DAgger needs a live, queryable expert during training, which is not always available or safe.

Page 14 — Why Offline RL?

Having just seen that online correction (DAgger) fixes compounding error but requires an expert available live, this slide sets up the alternative: what if we instead squeeze more out of a *fixed*, already-collected dataset, using reward labels instead of (or in addition to) an expert?

Mechanism

Online RL process	Offline RL process
Collect data	Given a static dataset
Update policy on the latest data (or all data so far)	Train the policy on the provided dataset only

Why offline RL can be attractive: it lets you reuse datasets already collected by people or by existing systems (previous experiments, robots, institutions) instead of paying to recollect; it avoids the risk of running an undertrained policy in the world to gather more data (unsafe in, e.g., medical or driving domains); and it doesn’t require writing down a reward function during data collection, only afterward for training. A common hybrid is offline pretraining followed by online fine-tuning, combining a safe, cheap starting point with a smaller, safer amount of live interaction.

Page 15 — Offline RL: Formal Setup

This section restates the RL objective (maximize expected return) but makes the data-generating process explicit: the dataset now comes from an unknown, possibly suboptimal, possibly mixed *behavior policy*, and unlike imitation learning, we assume we also have reward labels r attached to every transition.

Mechanism

$$\mathcal{D} : \{(s, a, s', r)\} \sim \pi_\beta, \quad p_{\pi_\beta}(\tau) = p(s_1) \prod_{t=1}^T \pi_\beta(a_t | s_t) p(s_{t+1} | s_t, a_t),$$

$$\max_{\theta} \mathbb{E}_{p_{\pi_\theta}} \left[\sum_t r(s_t, a_t) \right].$$

Symbol	What it means
π_β	the (unknown) <i>behavior policy</i> that generated the dataset
$p_{\pi_\beta}(\tau)$	the probability of an entire trajectory τ under π_β and the environment dynamics
$p(s_1)$	the distribution of starting states
$p(s_{t+1} s_t, a_t)$	the environment’s (unknown, fixed) transition dynamics
$r(s_t, a_t)$	the reward received for taking action a_t in state s_t

Two things are new relative to imitation learning: (1) we now have reward labels r , so “good” is defined numerically rather than purely by what the expert happened to do, and (2) π_β is explicitly allowed to be a *mixture* of policies (data from many different sources, of many different skill levels) rather than one consistent expert. The objective itself — maximize expected return under the policy we’re learning, π_θ — is the same objective as ordinary (online) RL; what changes is that we are only allowed to use $\mathcal{D}$, never new rollouts, to solve it.

Page 16 — Offline RL versus Imitation Learning: Trajectory Stitching

Even with reward labels available, why bother with RL machinery instead of just imitating the whole dataset? Because a dataset can contain *no single trajectory* that is good all the way through, while still containing all the pieces needed to construct one.

Mechanism

The diagram shows two overlapping recorded trajectories that pass through a shared state. One continuation from that shared state reaches a rewarding state s_9 (with $r(s_9) = +1$); a different continuation from the very same shared state instead wanders to a less useful state s_6 (with $r(s_6) = -1$). Neither full recorded trajectory, taken as a whole, is uniformly good — but the *first half* of one trajectory (getting to the shared state) combined with the *second half* of the other (continuing on to s_9) would be. Behavior cloning can only ever reproduce a trajectory it saw; it has no mechanism for splicing together the good part of one trajectory with the good part of another. Offline RL, by contrast, learns state-indexed values (via Bellman backups) rather than whole-trajectory imitation: at the shared state, whichever continuation has higher backed-up value gets favored regardless of which original trajectory it came from, effectively **stitching** the good first half to the good second half into a policy better than either recorded trajectory alone.

Reference expansion. This “stitching” property of value-based (Bellman-backup) methods versus trajectory-imitation methods is discussed explicitly in Levine, Kumar, Tucker & Fu, *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems* (arXiv:2005.01643, 2020): imitation-style methods are fundamentally limited by the return of the single best trajectory (or best sub-population) actually present in the data, whereas dynamic-programming-style

offline RL methods can in principle exceed any individual trajectory’s return by recombining sub-trajectories through the state-value function. The bracketed questions on the slide point at this directly: a policy learned with vanilla imitation would just reproduce whichever recorded path it was shown (never reaching s_9 if that specific full path was never demonstrated), while a correctly-learned RL policy should be able to reach s_9 by stitching; whether **TD** or **Monte Carlo** value estimation is used matters here because MC value estimates are tied to the return of the specific trajectory they were sampled from, while TD backups propagate value *state-by-state* and are exactly the mechanism that makes cross-trajectory stitching possible.

Page 17 — Why Not Just Run an Off-Policy Algorithm Directly?

Off-policy actor-critic methods like SAC already claim to work from a replay buffer of past data. So why not just point one at a fixed offline dataset and call it a day? This slide sets up the question; the next slide gives the answer.

Mechanism

The standard SAC-style Q-function update:

$$\min_{\phi} \sum_{(s,a,s') \sim \mathcal{D}} \left\| \hat{Q}_{\phi}^{\pi_{\theta}}(s, a) - (r(s, a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s', a')]) \right\|^2,$$

followed by a policy update using the reparameterized (or likelihood-ratio) policy gradient

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i^{\pi^*} | s_i) \hat{Q}_{\phi}^{\pi_{\theta}}(s_i, a_i^{\pi^*}), \quad a_i^{\pi^*} \sim \pi_{\theta}(a | s_i).$$

Symbol	What it means
$\hat{Q}_{\phi}^{\pi_{\theta}}(s, a)$	the learned critic’s estimate of the value of taking a in s under π_{θ}
γ	discount factor
$a' \sim \pi_{\theta}(\cdot s')$	the <i>current</i> policy’s own action at the next state, used for bootstrapping
$a_i^{\pi^*}$	an action freshly sampled from the current policy, used to compute the policy gradient

The critical detail buried in this formula: both the Q-update’s bootstrap target and the policy-gradient step evaluate $\hat{Q}_{\phi}$ at an action a' (or $a_i^{\pi^*}$) sampled from the *current* policy π_{θ} , not necessarily an action that ever appears in the fixed dataset $\mathcal{D}$. When new data keeps arriving (the online setting), any inaccuracy this introduces gets corrected the next time that action is actually tried in the environment. With a *static* dataset from a mediocre policy, that corrective feedback loop is gone — which is exactly the problem the next page dissects.

Page 18 — The Out-of-Distribution (OOD) Action Problem

This is the central reason naive off-policy algorithms fail offline: the critic gets evaluated at actions it was never trained on, its errors there are never corrected by real experience, and those errors get amplified rather than fixed by the Bellman backup.

Mechanism

Consider the same critic loss as Page 17. At a next-state s' , the target requires $\mathbb{E}_{a' \sim \pi_\theta(\cdot|s')}[\hat{Q}_\phi^{\pi_\theta}(s', a')]$ — an average over actions the *current* policy would choose, which may fall well outside the region of actions actually present in $\mathcal{D}$ for that state (the “data support”). The diagram plots a randomly-initialized $Q(s', a')$ as a function of a' : it is a wiggly, essentially arbitrary curve outside the narrow green data-support region, since it was never trained there. If the policy update is free to pick whichever a' maximizes this untrained curve, it will latch onto one of the curve’s random-noise peaks lying *outside* the data support — an action the critic simply has no real information about.

Symbol	What it means
data support	the range of actions actually observed in $\mathcal{D}$ for a given state
OOD action	an action fed to Q that lies outside that observed range

This creates a feedback loop with no brake: (1) Q is unreliable (essentially noise) on OOD actions; (2) the policy update seeks out exactly those actions where Q happens to look over-optimistic; (3) the next Bellman backup then treats that already-inflated value as a real target and inflates Q further; (4) each round the learned policy drifts further from the behavior policy, chasing an ever more overestimated (and ever less trustworthy) Q -value. This phenomenon is documented in the offline RL literature as *extrapolation error* or the OOD-action overestimation problem, e.g., Fujimoto, Meger & Precup, *Off-Policy Deep Reinforcement Learning without Exploration* (BCQ, 2019) and Kumar, Zhou, Tucker & Levine, *Conservative Q-Learning for Offline Reinforcement Learning* (CQL, 2020).

Numerical Walkthrough

Suppose the true (unobservable) values for a state s' are $Q^*(s', a' = 1) = 5$ (in-support) and $Q^*(s', a' = 9) = 0$ (far out of support, a bad action). Before training converges, the randomly initialized network might output $\hat{Q}_\phi(s', 1) = 4.8$ (close to truth, since $a' = 1$ is well covered by data) but $\hat{Q}_\phi(s', 9) = 7.0$ (a spurious overestimate, since $a' = 9$ was never trained on). A policy update that samples candidate actions and reinforces whichever has higher $\hat{Q}_\phi$ will push probability mass toward $a' = 9$, because $7.0 > 4.8$ — even though the true value ordering is the exact opposite ($5 > 0$). The next Bellman backup then uses this newly-likely $a' = 9$ (with its inflated estimate 7.0) as part of the bootstrap target for other states, propagating the error outward through the whole value function.

Page 19 — Offline RL: A Simple Baseline (Filtered Behavior Cloning)

Before reaching for a full off-policy critic, here is the simplest possible way to use reward labels: throw away the worst trajectories and just imitate what is left.

Mechanism

1. Rank whole trajectories by total return: $r(\tau) = \sum_{(s_t, a_t) \in \tau} r(s_t, a_t)$.
2. Keep only the top-performing slice of the dataset: $\tilde{\mathcal{D}} : \{\tau \mid r(\tau) > \eta\}$, for some return threshold η .

3. Run ordinary behavior cloning (maximum likelihood) on the filtered set:
 $\max_{\theta} \sum_{(s,a) \in \tilde{\mathcal{D}}} \log \pi_{\theta}(a \mid s).$

Symbol	What it means
$r(\tau)$	total return of one whole trajectory
η	the return cutoff; only trajectories scoring above this survive
$\tilde{\mathcal{D}}$	the filtered dataset, containing only high-return trajectories

Numerical Walkthrough

Suppose $\mathcal{D}$ has five trajectories with returns $r(\tau) = \{2, 8, -1, 6, 4\}$, and we set $\eta = 5$ (roughly, keep the top 40%). Then $\tilde{\mathcal{D}} = \{\tau : r(\tau) \in \{8, 6\}\}$ — only the two best trajectories survive, and behavior cloning is run on just their (s, a) pairs, ignoring the rest of the dataset entirely.

Limitation. This baseline never combines partial trajectories the way stitching does (Page 16): it can only ever be as good as the single best *whole* trajectory that happened to be recorded, and it wastes all the reward information contained in the discarded lower-return trajectories (which, per the stitching argument, might still contain a locally-excellent sub-path worth reusing).

Page 20 — Weighted Imitation Learning

Instead of a hard cutoff (keep this trajectory / discard that one), why not softly weight *every single transition* by how good that specific action was, using a continuous score rather than a whole-trajectory return?

Mechanism

The natural per-transition “how good was this action” score is the advantage function, recalled from earlier in the course:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

Weighted imitation then reweights the log-likelihood objective by (an increasing function of) this advantage:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s) \exp(A(s, a))].$$

Symbol	What it means
$Q^{\pi}(s, a)$	expected return of taking a in s , then following π thereafter
$V^{\pi}(s)$	expected return of state s , averaged over π ’s own action choices
$A^{\pi}(s, a)$	how much better (or worse) a is than π ’s “typical” action in s
$\exp(A(s, a))$	converts the advantage into a positive weight > 0 for that (s, a) pair

Because $\exp(\cdot)$ is always positive and increasing, actions with a large positive advantage get an outsized weight in the log-likelihood sum, while actions with negative advantage get down-weighted toward (but never fully to) zero — so the policy is pulled toward reproducing the *better-than-typical* actions in the dataset and away from reproducing the worse ones, rather than treating every recorded action as equally worth imitating. This directly connects to Page 16’s stitching diagram: at the shared state, the branch of the graph that leads toward the rewarding state s_9 gets a large positive advantage (and hence a large weight, shown as the bold arrow in the “Adv.

weights for s_3 ” box), while the branch leading to the low-reward state s_6 gets a small or negative advantage (thin arrow) — so advantage weighting is precisely the mechanism that accomplishes the “stitching” described qualitatively on Page 16.

Page 21 — Advantage-Weighted Regression (AWR)

This slide fills in *how* to actually get $V(s)$ and how to keep the exponential weight from Page 20 numerically well-behaved, turning the idea into a full, runnable algorithm. Source: Peng, Kumar, Zhang & Levine, *Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning* (arXiv:1910.00177, 2019), as linked directly on the slide.

Mechanism

Step 1 — fit a value function by Monte Carlo regression onto the empirical (observed) return-to-go:

$$\min_{\phi} \sum_{s_t \sim \mathcal{D}} \left\| \hat{V}_{\phi}^{\pi_{\beta}}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right\|^2.$$

Step 2 — train the policy with an advantage-weighted log-likelihood, using a temperature α inside the exponent:

$$\max_{\theta} \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\log \pi_{\theta}(a_t | s_t) \exp \left(\frac{1}{\alpha} \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi_{\beta}}(s_t) \right) \right) \right].$$

Symbol	What it means
$\hat{V}_{\phi}^{\pi_{\beta}}(s_t)$	the fitted value function, an estimate of how good s_t is under the <i>behavior</i> policy π_{β}
$\sum_{t'=t}^T r(s_{t'}, a_{t'})$	the empirical (Monte Carlo) return-to-go actually observed after s_t in the data
α	a temperature hyperparameter dividing the advantage before exponentiating

The quantity inside the exponent, $\sum_{t'=t}^T r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi_{\beta}}(s_t)$, is exactly a Monte Carlo estimate of the advantage $A^{\pi_{\beta}}(s_t, a_t)$ from Page 20 (observed return minus the value baseline). α exists purely for numerical stability: because $\exp(\cdot)$ grows extremely fast, dividing the advantage by α before exponentiating keeps the weights from blowing up to numbers too large for stable gradient updates — a larger α flattens (softens) the weighting, a smaller α sharpens it toward favoring only the very best actions.

Note on the slide’s return formula. The formula on the slide sums undiscounted rewards $\sum_{t'=t}^T r(s_{t'}, a_{t'})$; the original AWR paper’s more general presentation uses a discounted return, $\sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}, a_{t'})$. The undiscounted version shown here is simply the finite-horizon (episodic) special case with $\gamma = 1$ and is not a mistake, just a simplification worth being aware of.

Answering the slide’s question (“what do you learn for deterministic π_{β} ?”): if the behavior policy was in fact deterministic, then for every state s_t actually visited in the data there is exactly one recorded action, so the fitted $\hat{V}_{\phi}^{\pi_{\beta}}(s_t)$ and the single observed return-to-go tend to coincide, driving the advantage estimate toward zero for every recorded transition (weight $\exp(0/\alpha) = 1$ for all of them) — AWR then degenerates toward plain behavior cloning, since there is no variation in behavior at a given state to reward-weight between.

Page 22 — Monte Carlo Estimation: Trade-offs

A short pros/cons pause before moving to a TD-based alternative on the next page.

Advantages	Disadvantages
Simple: just sum up observed rewards, no bootstrapping equations to get wrong.	Monte Carlo estimates are noisy: a single sampled trajectory's return has high variance, especially in stochastic environments or with long horizons.
Avoids ever querying or training on any out-of-distribution action (Page 18's problem never arises), because the target is built purely from rewards that were actually observed.	$\hat{A}^{\pi_\beta}$ estimates the advantage of the <i>behavior</i> policy π_β , not of the (better) policy π_θ we're actually trying to learn — so we are optimizing against a weaker yardstick than we ultimately want.

The second disadvantage is the direct motivation for the next slide: can we estimate advantages with TD updates (which bootstrap off a Q-function that tracks the *current*, improving policy π_θ) instead of Monte Carlo (which is stuck estimating the advantage of the old data-collecting policy π_β)?

Page 23 — Advantage-Weighted Actor-Critic (AWAC)

AWAC replaces AWR's noisy, π_β -locked Monte Carlo advantage with a TD-estimated advantage that tracks the current policy π_θ — while trying to keep AWR's key safety property (never training the *policy* on out-of-distribution actions). Source: Nair, Gupta, Dalal & Levine, *AWAC: Accelerating Online Reinforcement Learning with Offline Datasets* (arXiv:2006.09359, 2020).

Mechanism

Step 1 — estimate a Q-function with a standard TD target:

$$\min_{\phi} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - (r + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot|s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s',a')]) \right)^2 \right], \quad \text{alternative: } a' \sim \mathcal{D}.$$

Step 2 — update the policy with AWR, using this TD-based advantage instead of a Monte Carlo one:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} [\log \pi_{\theta}(a | s) \exp(\hat{A}^{\pi_{\theta}}(s,a))], \quad \hat{A}^{\pi_{\theta}}(s,a) = \hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - \mathbb{E}_{\bar{a} \sim \pi_{\theta}(\cdot|s)} [\hat{Q}_{\phi}^{\pi_{\theta}}(s,\bar{a})].$$

Symbol	What it means
$\hat{A}^{\pi_{\theta}}(s,a)$	advantage estimated from the learned Q , using the <i>current</i> policy π_{θ} as the baseline
$\bar{a}$	a fresh action sampled from $\pi_{\theta}(\cdot s)$, used only to compute the baseline $\mathbb{E}[\hat{Q}(s,\bar{a})]$

Contrast to avoid confusing: which step actually queries OOD actions? There are *two* places an action from π_{θ} could be plugged into $\hat{Q}_{\phi}$, and they behave very differently:

- Step 2’s baseline, $\mathbb{E}_{\bar{a} \sim \pi_\theta(\cdot | s)}[\hat{Q}_\phi(s, \bar{a})]$, is subtracted off inside the advantage, but the policy’s log-likelihood term $\log \pi_\theta(a | s)$ in that same objective is always evaluated at an action *a taken from the dataset* $\mathcal{D}$ — so the policy itself is never directly trained to increase probability on a self-sampled, possibly OOD action. This is the “+” marked on the slide: “still only trains policy on actions in the dataset.”
- Step 1’s TD target, however, does bootstrap through $\hat{Q}_\phi(s', a')$ with $a' \sim \pi_\theta(\cdot | s')$ — exactly the OOD query pattern flagged as dangerous on Page 18. This is the “−” marked on the slide: “queries on OOD actions.” The slide notes an alternative that avoids even this: sample $a' \sim \mathcal{D}$ instead (i.e., use the actually-recorded next action, a SARSA-style target), fully sidestepping any OOD query in the critic as well.

The “+” relative to plain AWR is that $\hat{Q}_\phi^{\pi_\theta}$ tracks the value of the policy we are currently improving, π_θ , rather than being frozen at the value of the old data-collecting policy π_β the way AWR’s Monte Carlo advantage was (Page 22’s second disadvantage) — so AWAC gets a better-targeted advantage signal, at the cost of reintroducing a (milder, critic-only) version of the OOD problem.

Page 24 — Motivating Expectile Regression

Both AWR and AWAC still bootstrap through the policy’s own action distribution somewhere. Can we get a TD-style value estimate that is optimistic (biased toward the best available action) *without ever evaluating Q at an action outside the dataset at all?*

Mechanism

The slide’s histogram plots $\mathbb{E}_{a \sim \pi_\beta(\cdot | s)}[Q(s, a)]$ across the range of actions actually taken by the behavior policy at state s (the data support). An ordinary squared-error (ℓ_2) fit of $V(s)$ to this distribution recovers its *mean* — the average value of a typical behavior-policy action, marked by the left dashed line. But what we actually want for $V(s)$ is closer to the value of the *best* action still contained within the data support (the right dashed line, further right / higher) — an optimistic-but-in-support estimate, without ever having to plug in an action outside that histogram (which is exactly the OOD danger from Page 18). The question posed is whether a different loss function than plain ℓ_2 can shift the fitted value from “the mean” toward “a high percentile” of this same in-support distribution.

Page 25 — Expectile Regression

Ordinary least-squares gives you the mean of a distribution because it penalizes positive and negative errors identically. Expectile regression breaks that symmetry, so the minimizer shifts away from the mean toward a chosen upper (or lower) percentile-like statistic — the “expectile.”

Mechanism

$$\ell_2^\lambda(x) = \begin{cases} (1 - \lambda)x^2, & x < 0, \\ \lambda x^2, & \text{otherwise.} \end{cases}$$

Symbol	What it means
x	the residual (predicted value minus target, or vice versa, depending on context)
λ	the asymmetry parameter, $\lambda \in (0, 1)$, controlling which side is penalized more

When $\lambda = 0.5$, both branches have weight 0.5 and this is exactly ordinary ℓ_2 loss, whose minimizer is the mean. When $\lambda \neq 0.5$, one side of the residual is penalized more heavily than the other, so the loss is minimized not at the mean but at a value shifted toward whichever side is penalized *less* (since the fit “gives way” more easily on the cheap side and resists more on the expensive side). The plot on the slide shows this directly: for $\lambda = 0.01$ the loss curve is nearly flat for $x < 0$ and steep for $x > 0$, and for $\lambda = 0.99$ it’s the mirror image.

Numerical Verification

Take a toy distribution of $Q(s, a)$ values over four in-support actions, each equally likely: $\{2, 4, 6, 8\}$ (true mean = 5). Define the residual as in the next slide’s convention, $x = V(s) - Q(s, a)$, and numerically minimize $\mathbb{E}[\ell_2^\lambda(x)]$ over candidate values of $V(s)$ (grid search):

λ	minimizing $V(s)$
0.01	7.88 (close to the max, 8)
0.10	7.00
0.50	5.00 (exactly the mean)
0.90	3.00
0.99	2.12 (close to the min, 2)

This confirms, with actual numbers, that *small* λ pushes $V(s)$ toward the *maximum* of the in-support Q -distribution, and $\lambda = 0.5$ exactly reproduces the ordinary mean. This is the reverse of the residual convention used in most published expectile-regression write-ups (which define the residual as target-minus-prediction, $Q - V$, so that a large parameter close to 1 gives the high expectile) — but because this slide’s loss uses the residual $V(s) - Q(s, a)$ (predicted minus target, the opposite sign), the direction flips, and it is *small* λ that yields the high, near-maximum expectile. This is exactly why Page 26 specifies “using small $\lambda < 0.5$ ” to make $\hat{V}(s)$ approximate the best in-support action’s value, matching the goal illustrated by Page 24’s histogram.

Page 26 — Implicit Q-Learning (IQL)

This slide assembles the full offline algorithm that achieves Page 24’s goal: an optimistic, in-support value estimate, obtained entirely from TD updates, without ever plugging an out-of-distribution action into Q or V .

Mechanism

Step 1 — fit $\hat{V}(s)$ with the expectile loss, comparing it only to Q -values of actions actually in the dataset:

$$\hat{V}(s) \leftarrow \arg \min_V \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\ell_2^\lambda(V(s) - \hat{Q}(s, a)) \right], \quad \text{using small } \lambda < 0.5.$$

Step 2 — update Q with an ordinary MSE / TD loss, but bootstrapping off $\hat{V}(s')$ (a function of the *state* s' only) rather than off $Q(s', a')$ evaluated at some sampled next action:

$$\hat{Q}(s, a) \leftarrow \arg \min_Q \mathbb{E}_{(s, a, s') \sim \mathcal{D}} \left[(Q(s, a) - (r + \gamma \hat{V}(s')))^2 \right].$$

Step 3 — extract a policy with AWR, exactly as on Page 21/23:

$$\hat{\pi} \leftarrow \arg \max_{\pi} \mathbb{E}_{s, a \sim \mathcal{D}} \left[\log \pi(a | s) \exp \left(\frac{1}{\alpha} (\hat{Q}(s, a) - \hat{V}(s)) \right) \right].$$

Symbol	What it means
$\hat{V}(s)$	a state-only value function, trained to be an optimistic, in-support estimate
$\hat{Q}(s, a)$	the Q-function, trained with ordinary TD regression, bootstrapping through $\hat{V}$

Why this closes the OOD loophole for good. Compare this directly with AWAC (Page 23), whose one remaining weakness was that its Q -update needed $a' \sim \pi_{\theta}(\cdot | s')$ (an action sampled from the current policy, potentially outside the data). Here, Step 2’s target is $r + \gamma \hat{V}(s')$ — and because $\hat{V}$ takes only a *state* as input, computing it never requires sampling or specifying *any* action at s' at all. Step 1, meanwhile, only ever evaluates $\hat{Q}(s, a)$ at pairs (s, a) literally drawn from $\mathcal{D}$. So across the whole algorithm, Q is never queried at an action it wasn’t trained on, and the “implicit” in the name refers to exactly this trick: the effect of a max over actions (needed, informally, to be optimistic) is achieved *implicitly* through the asymmetric expectile loss on in-support data, rather than through an explicit $\max_a$ or a sampled $a' \sim \pi_{\theta}$.

Page 27 — References

The slide deck credits Prof. Chelsea Finn’s Stanford CS 224R lectures on Imitation Learning and Offline RL as the source material, with links to the corresponding recorded lectures on YouTube. The paper-level references used for the expansions above are: Ross & Bagnell (2010) and Ross, Gordon & Bagnell (2011) for the compounding-error/DAGger bounds; Chen et al. (2021, arXiv:2106.01345) for the Decision Transformer; Lipman et al. (2022) for flow matching and Chi et al. (2023, arXiv:2303.04137) for the original DDPM-based Diffusion Policy; Levine, Kumar, Tucker & Fu (2020, arXiv:2005.01643) for the offline-RL stitching argument; Fujimoto et al. (2019) and Kumar et al. (2020) for the OOD-action extrapolation-error problem; Peng, Kumar, Zhang & Levine (2019, arXiv:1910.00177) for AWR; Nair, Gupta, Dalal & Levine (2020, arXiv:2006.09359) for AWAC; and Kostrikov, Nair & Levine (2021, arXiv:2110.06169) for IQL.

Key Formulas

- Behavior cloning: $\min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(s, a) \in \mathcal{D}} \|a - \pi_{\theta}(s)\|^2$.
- Maximum-likelihood imitation: $\min_{\theta} -\mathbb{E}_{(s, a) \in \mathcal{D}} [\log \pi_{\theta}(a | s)]$.
- GMM density: $p(a | s) = \sum_{k=1}^K w_k(s) \mathcal{N}(a | \mu_k(s), \Sigma_k(s))$.
- Autoregressive factorization: $\pi_{\theta}(a | s) = \prod_{d=1}^D \pi_{\theta}(a_d | s, a_{<d})$.
- Flow-matching training loss: $\|v_{\theta}(x_t, t) - (x_1 - x_0)\|^2$, with $x_t = tx_1 + (1 - t)x_0$.

- Behavior-cloning error bound: $J(\pi^*) - J(\pi) = O(\epsilon T^2)$; DAgger: $O(\epsilon T)$.
- DAgger update: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s', a^*)\}$, $a^* \sim \pi_{\text{expert}}(\cdot \mid s')$.
- Offline RL objective: $\max_{\theta} \mathbb{E}_{p_{\pi_{\theta}}} [\sum_t r(s_t, a_t)]$ from data $\mathcal{D} \sim \pi_{\beta}$.
- Advantage function: $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$.
- Weighted / advantage-weighted imitation: $\arg \max_{\theta} \mathbb{E}_{s, a \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s) \exp(A(s, a)/\alpha)]$.
- AWR value fit: $\min_{\phi} \sum_{s_t} \|\hat{V}_{\phi}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'})\|^2$.
- Off-policy TD critic target: $r(s, a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot \mid s')} [\hat{Q}_{\phi}(s', a')]$.
- Expectile loss: $\ell_2^{\lambda}(x) = (1 - \lambda)x^2$ if $x < 0$, else λx^2 .
- IQL value fit: $\hat{V}(s) \leftarrow \arg \min_V \mathbb{E}_{(s, a) \sim \mathcal{D}} [\ell_2^{\lambda}(V(s) - \hat{Q}(s, a))]$, small $\lambda < 0.5$.
- IQL Q fit: $\hat{Q}(s, a) \leftarrow \arg \min_Q \mathbb{E}_{(s, a, s') \sim \mathcal{D}} [(Q(s, a) - (r + \gamma \hat{V}(s')))^2]$.
- IQL policy extraction: $\hat{\pi} \leftarrow \arg \max_{\pi} \mathbb{E}_{s, a \sim \mathcal{D}} [\log \pi(a \mid s) \exp(\frac{1}{\alpha}(\hat{Q}(s, a) - \hat{V}(s)))]$.